

1. Conceitos base

Cada tarefa tem:

- **C** = tempo de execução
- **T** = período (quando volta a aparecer)
- **D** = deadline (prazo)

A cada período surge uma nova instância da tarefa.

2. Escalonamento com prioridades fixas

Rate Monotonic (RM)

- Prioridade baseada no **período (T)**
- **Menor T $\Rightarrow$ maior prioridade**

Deadline Monotonic (DM)

- Prioridade baseada no **deadline (D)**
- **Menor D $\Rightarrow$ maior prioridade**

Ambos são **preemptivos**:

- Uma tarefa **pode interromper outra** se tiver maior prioridade

3. EDF (Earliest Deadline First)

- Prioridade dinâmica
- Quem tiver **deadline mais próximo executa**

Também normalmente **preemptivo**

4. Multi-processador

Global EDF

- Todas as tarefas numa fila global
- São distribuídas dinamicamente pelos CPUs
- Pode haver **migração de tarefas**

Partitioned EDF

- Tarefas são **fixas a um CPU**
- Depois cada CPU usa EDF localmente
- Não há migração

5. Recursos partilhados

Problema:

- Tarefas competem por recursos (R $\rightarrow$ lock, V $\rightarrow$ unlock)
- Pode haver **inversão de prioridades**

PIP (Priority Inheritance)

- Tarefa com baixa prioridade **herda prioridade alta** temporariamente

- Resolve bloqueios

ICPP (Ceiling)

- Cada recurso tem um teto de prioridade
- Quando uma tarefa entra no recurso → assume logo prioridade alta
- Evita conflitos antecipadamente

Preempção vs Não-preempção

Tipo	O que acontece
Preemptivo	Tarefas podem interromper-se
Não-preemptivo	Tarefa acaba antes de outra começar

Resumo geral

1. Tipos de Sistemas

Real-Time OS (RTOS)

- Tem **deadlines obrigatórias**
- Tipos:
 - **Hard** → falha = sistema quebra
 - **Soft** → atraso degrada desempenho
 - **Firm** → resultado fora do tempo não vale

2. Escalonamento (Scheduling)

a) Fixed Priority (FP)

- Prioridade fixa

RM (Rate Monotonic)

- menor **período** → maior prioridade

DM (Deadline Monotonic)

- menor **deadline** → maior prioridade

b) Dynamic Priority

EDF (Earliest Deadline First)

- prioridade = **deadline mais próximo**
- melhor utilização CPU
- problema: overload imprevisível

LLF (Least Laxity First)

- escolhe tarefa com **menos folga (slack)**

TESTES IMPORTANTES

- EDF:
 $U \leq 1 \rightarrow$ escalonável
- FP (RM):
limite $\approx n(2^{1/n} - 1)$

Preempção

Tipo	Significado
 Preemptivo	tarefa pode ser interrompida
 Não-preemptivo	tarefa termina primeiro

 EDF, RM, DM $\rightarrow$ normalmente **preemptivos**

3. Tipos de tarefas

- **Periódicas** $\rightarrow$ repetem-se (T)
- **Esporádicas** $\rightarrow$ aparecem raramente
- **Aperiódicas** $\rightarrow$ sem padrão [\[Binder1.pdf | PDF\]](#)

4. Servidores (tarefas aperiódicas)

Servidores = tarefa especial com:

- Período (T_s)
- Capacidade (Q_s)

Tipos principais

Polling Server

- executa mesmo sem pedidos
- simples

Deferrable Server

- guarda capacidade $\rightarrow$ responde melhor

Sporadic Server

- mantém limites $\rightarrow$ melhor para análise

TBS / CBS

- usados com EDF
- CBS controla utilização (mais seguro)

5. Multiprocessador

Tipos:

- **Global** tarefas podem mudar CPU
- **Partitioned** cada tarefa fixa num CPU

Problemas:

- **Dhall effect**

- **bin packing (NP-hard)**

6. Recursos Partilhados

Problemas:

- **Deadlock**
- **Priority inversion**

 prioridade alta espera por baixa

Protocolos

PIP

- tarefa baixa herda prioridade

Ceiling (ICPP)

- sobe prioridade logo ao entrar no recurso

SRP

- funciona com prioridades dinâmicas

7. Processos e Threads

Processo

- programa em execução
- tem memória própria

Thread

- parte do processo
- partilha memória
- mais leve

Estados do processo

- Ready
- Running
- Waiting
- Terminated

Context Switch

- troca entre processos
- guarda estado (PCB)

8. Sincronização

Semáforos

- contador
- operações:

- wait (↓)
- signal (↑)

◆ Mutex

- apenas 1 tarefa entra
- mais eficiente que semáforo

◆ Spinlock

- espera ativa (não dorme)

◆ Wait queue

- tarefa fica à espera de evento

◆ Atomic

- operações sem interrupção

⚠ PROBLEMAS IMPORTANTES

Race condition

- duas tarefas acedem ao mesmo recurso

Deadlock

- tarefas bloqueadas entre si

Starvation

- tarefa nunca recebe CPU

🧩 9. Conceitos de OS

OS faz: gestão de CPU; memória; processos; I/O

Kernel

- parte principal do OS
- gere hardware

Modos:

- User mode
- Kernel mode

🧠 10. MEMÓRIA

- Virtual Memory → ilusão de memória infinita
- Page table → mapeia virtual → física

Exemplos de perguntas:



1. Escalonamento

Q1 No algoritmo Rate Monotonic:

- (a) A tarefa com menor deadline tem maior prioridade
- (b) A tarefa com menor período tem maior prioridade
- (c) A tarefa com maior execução tem maior prioridade
- (d) Todas estão corretas

Q2 No algoritmo EDF:

- (a) Prioridades são fixas
- (b) A tarefa com deadline mais próximo executa
- (c) A tarefa com maior período executa
- (d) Não há preempção

Q3 Em EDF, o sistema é escalonável se:

- (a) $U > 1$
- (b) $U = 1$
- (c) $U \leq 1$
- (d) $U \geq 1$

Q4 A principal diferença entre RM e DM é:

- (a) RM usa deadlines
- (b) DM usa período
- (c) RM usa período e DM usa deadline
- (d) São iguais



2. Multiprocessador

Q5 Num sistema multiprocessador:

- (a) Não há paralelismo
- (b) Existe paralelismo
- (c) Só executa uma tarefa
- (d) Nenhuma

Q6 No Global Scheduling:

- (a) tarefas não mudam de CPU
- (b) tarefas podem migrar entre CPUs
- (c) cada CPU tem tarefas fixas
- (d) não existe fila global

Q7 No Partitioned Scheduling:

- (a) tarefas podem migrar
- (b) tarefas são fixas num CPU
- (c) EDF não pode ser usado
- (d) não há prioridades

3. Recursos Partilhados

Q8 Priority inversion ocorre quando:

- (a) tarefa baixa espera por alta
- (b) tarefa alta espera por baixa
- (c) tarefas não executam
- (d) CPU fica livre

Q9 No PIP:

- (a) a tarefa de alta prioridade herda prioridade baixa
- (b) a tarefa de baixa prioridade herda prioridade alta
- (c) não há alteração de prioridades
- (d) todas estão erradas

Q10 No protocolo ICPP:

- (a) prioridade aumenta só quando há bloqueio
- (b) prioridade aumenta ao entrar no recurso
- (c) não evita deadlocks
- (d) ignora recursos

4. Processos e Threads

Q11 Um processo é:

- (a) programa em execução
- (b) apenas código
- (c) apenas dados
- (d) thread

Q12 Uma thread:

- (a) tem memória própria
- (b) partilha memória com outras threads
- (c) é mais pesada que um processo
- (d) não usa CPU

Q13 Context switch é:

- (a) mudança de programa
- (b) troca de estado entre tarefas

- (c) execução de código
- (d) criação de processo

5. Sincronização

Q14 Semáforos:

- (a) são apenas booleanos
- (b) são contadores
- (c) só funcionam em threads
- (d) não bloqueiam

Q15 Mutex:

- (a) permite acesso múltiplo
- (b) só permite uma tarefa
- (c) é igual a semáforo
- (d) não bloqueia

Q16 Spinlock:

- (a) bloqueia e dorme
- (b) usa espera ativa
- (c) liberta CPU sempre
- (d) é igual mutex

6. Problemas clássicos

Q17 Deadlock:

- (a) tarefas executam rápido
- (b) tarefas ficam bloqueadas entre si
- (c) CPU está livre
- (d) não há erro

Q18 Race condition:

- (a) problema de tempo
- (b) acesso simultâneo a dados
- (c) erro de memória
- (d) erro de CPU

7. Sistema Operativo

Q19 O sistema operativo:

- (a) gere recursos
- (b) fornece interface
- (c) abstrai hardware

- (d) todas

Q20 Kernel:

- (a) é programa do utilizador
- (b) é parte central do OS
- (c) não gere hardware
- (d) é opcional